

Smart Barangay AI

Requirements Audit Report

Audit date: 31 August 2026
Audited branch: `fix/sync-admin-resident-supabase`, synchronized with `origin/main` at `0d23a14`
Scope: Resident PWA, Admin Portal, API, and Supabase schema
Assessment rule: A feature is considered existing only when it is persisted and works end-to-end, not merely displayed in a mock or static screen.

5

Partial / broken

1

Missing

1

Implemented

Requirement Matrix

Requirement	Status	Evidence and audit finding
Chatbot: dynamic form	Partial	The chatbot detects document keywords and opens an in-chat form. The form only collects a purpose/notes field. It has no dynamic service-specific fields, document uploads, requirements validation, or structured form data. <code>apps/resident-pwa/components/chat/InChatFormCard.tsx:22-53</code>
Admin receives requests; one transaction/chatbot session for multiple requests	Partial / broken in real DB	The Admin Portal has a request queue and status actions. However, requests are not grouped to a chatbot session or transaction. The frontend creates a session ID but saves chat messages with <code>session_id: null</code> . The request form submits the auth user ID as <code>resident_id</code> , while the database expects a residents-table ID. <code>apps/resident-pwa/app/(app)/requests/new/page.tsx:52-61</code> <code>supabase_schema.sql:91-99</code>
Notify users when documents are ready for pickup, including payment requirements	Partial	A resident notification inbox exists and Admin can mark a request as Released. There is no automatic notification when a status changes. There are also no payment amount, due status, receipt, or payment-required fields in the request/notification model. <code>apps/admin-portal/app/(dashboard)/documents/actions.ts:55-90</code> <code>supabase_schema.sql:206-221</code>
Admin generates a filled template from chatbot data	Missing	The documents table only stores a file URL and generating user. No request form payload, document templates, field mapping, or PDF/DOCX generation workflow exists. <code>supabase_schema.sql:102-107</code>
Requests recorded for dashboard reports	Partial	Document requests are stored and the dashboard counts pending requests. Monthly transaction chart data is hard-coded, and the Reports page displays static mock reports rather than generating or downloading reports from database records. <code>apps/admin-portal/app/(dashboard)/dashboard/actions.ts:28-76</code>
User registration requires staff review, validation, and approval	Implemented	The latest main branch now links the auth user to the resident profile, saves the address, assigns the Resident role, restricts pending/rejected accounts, loads pending residents from the database, supports staff-permission checks, approve/reject actions, rejection reasons, audit logging, and realtime queue refresh. The new migration must be applied to the deployed Supabase project. Applicant notifications after approval/rejection are still absent. <code>apps/resident-pwa/components/auth/RegisterForm.tsx:61-137</code> <code>apps/admin-portal/app/(dashboard)/residents/actions.ts:141-268</code> <code>supabase/migrations/001_registration_approval_rbac.sql:1-204</code>
Clickable predefined questions only when answers are available	Partial	Five welcome buttons are hard-coded. They are not dynamically filtered against available knowledge or answerability. The knowledge base includes additional topics such as Business Clearance and Events, but those are not included in the buttons. <code>apps/resident-pwa/components/chat/ChatInterface.tsx:363-373</code> <code>apps/resident-pwa/lib/ai/policy-knowledge.ts:12-64</code>

Highest-Priority Missing Work

- Fix the authenticated-user to resident-profile relationship and use the correct resident ID for requests.
- Implement a structured dynamic chatbot request form with service-specific fields, uploads, validation, and saved payloads.

3. Persist chatbot sessions and link multiple document requests to one session/transaction.
4. Add payment fields and automatic resident/admin notifications for approval, payment, and pickup readiness.
5. Connect registration review actions to the database, including validation, approval/rejection reason, and applicant notification.
6. Implement template generation and real dashboard/report queries using stored request data.

Validation Performed

Resident PWA TypeScript check: passed before the branch synchronization.

Admin Portal TypeScript check: passed before the branch synchronization.

Branch synchronization: fast-forwarded to `origin/main`; local work was restored and four conflicts were resolved while preserving both sides.

Type checking does not detect the runtime foreign-key mismatch or static/mock-only behavior identified in this audit.

Generated from the Smart Barangay AI repository audit. This report describes the current code state and does not modify application functionality.